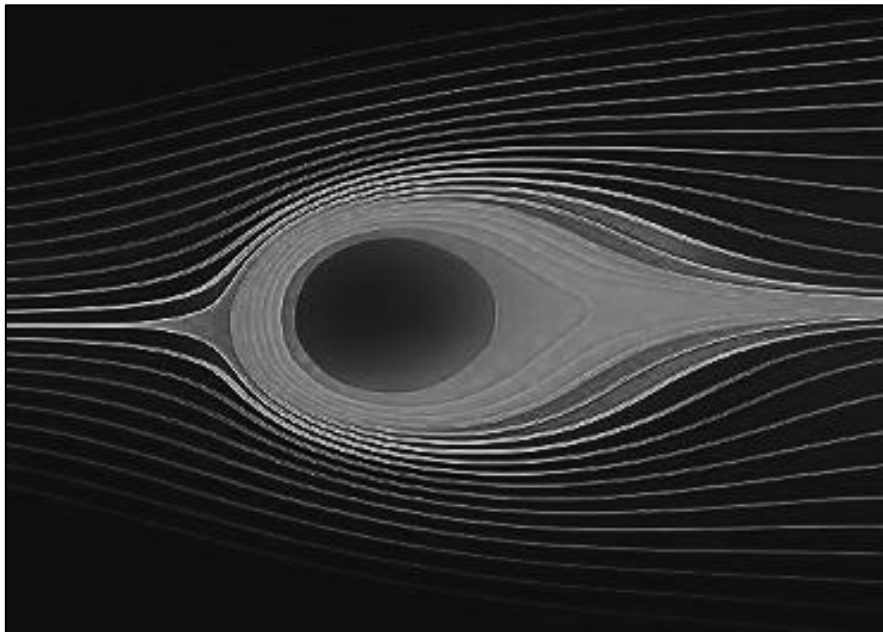


Επίλυση Ασκήσεων Μαθήματος

Υπολογιστική Ρευστομηχανική

1^η Σειρά Ασκήσεων



Ονοματεπώνυμο: Αναστάσιος Μπατατέγας

Αριθμός Μητρώου: 09121431

Εξάμηνο: 9^ο

Σχολή: ΕΜΦΕ

Email: tasosmntdew@gmail.com , ge21431@mail.ntua.gr

Διδάσκων: Ε. Ρετσίνης

Άσκηση 1.1

Υλοποιήστε στην γλώσσα προγραμματισμού Matlab τον αλγόριθμο Thomas για την επίλυση του παρακάτω τριδιαγώνιου συστήματος.

$$\begin{aligned} 2x_1 - x_2 &= 1 \\ x_1 + 2x_2 - x_3 &= 2 \\ 2x_2 + 4x_3 - x_4 &= -1 \\ 2x_4 - x_5 &= -2 \\ x_4 + 2x_5 &= -1 \end{aligned} \Leftrightarrow \begin{bmatrix} 2 & -1 & 0 & 0 & 0 \\ 1 & 2 & -1 & 0 & 0 \\ 0 & 2 & 4 & -1 & 0 \\ 0 & 0 & 0 & 2 & -1 \\ 0 & 0 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ -1 \\ -2 \\ -1 \end{bmatrix}$$

Λύση

Προς επίλυση του προβλήματος, μετασχηματίζουμε αρχικά το δεδομένο σύστημα σε μορφή πινάκων, όπως σημειώνεται και στην εκφώνηση, ήτοι στην μορφή:

$$\begin{bmatrix} 2 & -1 & 0 & 0 & 0 \\ 1 & 2 & -1 & 0 & 0 \\ 0 & 2 & 4 & -1 & 0 \\ 0 & 0 & 0 & 2 & -1 \\ 0 & 0 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ -1 \\ -2 \\ -1 \end{bmatrix} \quad (1.1)$$

όπου γίνεται εμφανές το γεγονός ότι ο πίνακας του συστήματος είναι τριδιαγώνιος, και επομένως ο αλγόριθμος Thomas μπορεί να εφαρμοστεί προς επίλυσή του.

Στην συνέχεια, υλοποιείται σε γλώσσα Matlab ο αλγόριθμος Thomas, ακολουθώντας την αλγοριθμική διαδικασία των Σημειώσεων του μαθήματος. Ο πηγαίος κώδικας γράφεται, και αποθηκεύεται, σε ένα αρχείο Matlab συνάρτησης (m-function file) με όνομα “ThomasAlgorithm.m”, και παρατίθεται παρακάτω. Η συνάρτηση δέχεται ως παραμέτρους τον τετραγωνικό πίνακα, A, του συστήματος, καθώς και το διάνυσμα των σταθερών, d, και επιστρέφει το διάνυσμα x, την λύση του συστήματος. Αρχικά, μετά την αρχικοποίηση του διανύσματος λύσης, x, και μετά τον ορισμό της διάστασης του συστήματος (μεταβλητή n), γίνεται έλεγχος της ορθότητας μορφής των δεδομένων εισόδου. Συγκεκριμένα, το διάνυσμα σταθερών, d, μετατρέπεται (αν δεν είναι ήδη) σε μορφή στήλης, και αξιοποιώντας την εντολή `size(A)`, ελέγχεται εάν οι διαστάσεις των εμπλεκόμενων πινάκων είναι ίσες (οπότε το σύστημα είναι «τετραγωνικό»). Επιπλέον, ελέγχεται το κατά πόσο ο εισαγόμενος πίνακας A είναι τριδιαγώνιος, με την εντολή `isbanded(A,1,1)`, η οποία επιστρέφει λογική τιμή 1 ή 0, ανάλογα με το αν ο πίνακας που εισάγεται ως πρώτο της όρισμα έχει «εύρος ζώνης» (bandwidth), του οποίου το κατώτερο και ανώτερο άκρο ορίζεται από το δεύτερο και τρίτο όρισμά της, αντιστοίχως (κάτω/άνω εύρος ζώνης = 1 ισοδυναμεί ότι μέχρι και η 1^n κάτω/άνω διαγώνιος του πίνακα είναι μη – μηδενικό διάνυσμα). Σε περίπτωση που κάποια απαραίτητη συνθήκη για να είναι το πρόβλημα καλώς ορισμένο δεν πληρείται, το πρόγραμμα σταματά, και επιστρέφεται το αντίστοιχο μήνυμα σφάλματος στο prompt window, με την αντίστοιχη εντολή `error()`, με προφανή λειτουργία. Μετά τον έλεγχο ορθότητας των δεδομένων εισόδου, η συνάρτηση υλοποιεί τον αλγόριθμο Thomas. Προς αυτό τον σκοπό, οι τρεις εμπλεκόμενες μη – μηδενικές διαγώνιοι του πίνακα A αντιγράφονται στα διανύσματα στήλης a,b και c, ακολουθώντας τον συμβολισμό του

αντίστοιχου εδαφίου των Σημειώσεων. Η αντιγραφή γίνεται με την αντίστοιχη εντολή `diag(A,k)`, η οποία επιστρέφει την k – οστή διαγώνιο του εισαγόμενου ως όρισμα πίνακα A . Τέλος, υλοποιούνται τα δύο στάδια του αλγορίθμου Thomas, ήτοι το στάδιο της εμπρόσθιας απαλοιφής, και το στάδιο της πίσω αντικατάστασης, και εντός των εμπλεκόμενων βρόγχων, γίνεται έλεγχος του μηδενισμού ή μη του εμπλεκόμενου παρονομαστή. Μηδενισμός του παρονομαστή σε κάποιο βήμα του βρόγχου σηματοδοτεί ότι το σύστημα έχει «κακή κατάσταση» (περίπτωση στην οποία ο παρονομαστής είναι «μηδέν» εντός της ακρίβειας της Matlab) ή ότι ο πίνακας του είναι μη – αντιστρέψιμος (περίπτωση όπου το σύστημα είναι ιδιάζον). Ο πλήρης πηγαίος κώδικας παρουσιάζεται παρακάτω.

MATLAB FUNCTION M-FILE (ThomasAlgorithm.m):

```
function [x] = ThomasAlgorithm(A,d)
% Function ThomasAlgorithm.m
%-----
% Solution of tridiagonal linear system A*x = d.
% Input : - square tridiagonal matrix A of coefficients (n x n).
%         - const. vector d (n x 1).
% Output: - vector x = solution of (Ax=d) (n x 1).
%-----

% --- System dimension & solution definition ---
n = length(d);
x = zeros(n,1);

% --- Input validity ---
% Ensure d is in column form :
d = d(:);
% Ensure A and d are properly sized :
[m1,m2]=size(A);
if(m1 ~= m2)
    error('Matrix A is not square (n x n).');
elseif(m1 ~= n)
    error('Matrix A and vector d are incompatible.');
```

```
end
% Ensure A is a tridiagonal matrix:
if(~isbanded(A,1,1))
    error('Matrix A is not tridiagonal.');
```

```
end

% --- Extract from A only the non-zero diagonals ---
% Form vector a (n x 1) of lower-diagonal elements:
a = zeros(n,1);
a(2:n) = diag(A,-1);
% Form vector b (n x 1) of diagonal elements:
b = zeros(n,1);
b = diag(A,0);
% Form vector c (n x 1) of upper-diagonal elements:
c = zeros(n,1);
c(1:n-1) = diag(A,1);

% --- Thomas' method implementation ---
% --- 1. Forward Elimination Stage ---
c(1) = c(1)/b(1);
d(1) = d(1)/b(1);
```

```
% Loop over vector c:
for i = 2 : n-1
    denominator = b(i)-a(i)*c(i-1);
    if(denominator==0)
        error('Matrix A is singular or has bad condition.');
```

```
end
    c(i) = c(i)/denominator;
end
% Loop over vector d:
for i = 2 : n
    denominator = b(i)-a(i)*c(i-1);
    if(denominator==0)
        error('Matrix A is singular or has bad condition.');
```

```
end
    d(i) = (d(i)-a(i)*d(i-1))/denominator;
end
x(n) = d(n);
% --- 2. Backward Substitution Stage ---
for i = n-1 : -1 : 1
    x(i) = d(i) - c(i)*x(i+1);
end
end
```

Σημειώνεται ότι το πρόγραμμα θα μπορούσε να τροποποιηθεί ώστε να λαμβάνει στην είσοδό του μόνον τρία διανύσματα, με στοιχεία αυτά των τριών μη – μηδενικών διαγωνίων του (τριδιαγώνιου) πίνακα του συστήματος. Ωστόσο, παρόλο που με αυτή την προσέγγιση δεν απαιτείται πολύπλοκη επεξεργασία και έλεγχος των δεδομένων εισόδου, με την πρακτική αυτή η ευθύνη ομαλής, ή μη, λειτουργίας της συνάρτησης επαφίεται σχεδόν εξολοκλήρου στον χρήστη (!).

Βάσει της παραπάνω συνάρτησης, η επίλυση του συστήματος γίνεται τετριμμένη, και παρόλο που μπορεί να γίνει απευθείας στο Command Window, παρουσιάζεται για εποπτικούς λόγους στο παρακάτω αρχείο (m-script file). Αρχικά, ορίζεται ο πίνακας A του συστήματος, στην συνέχεια το διάνυσμα d των σταθερών (το οποίο, λόγω του εξονυχιστικού ελέγχου των δεδομένων εισόδου στην παραπάνω υλοποίηση του αλγορίθμου Thomas δεν υπάρχει πρόβλημα να οριστεί ως γραμμή), και έπειτα η λύση του συστήματος, με εφαρμογή της συνάρτησης “ThomasAlgorithm(A,d)”, αποθηκεύεται στην μεταβλητή x , η οποία εκτυπώνεται στο τερματικό αφού δεν τοποθετείται “;” στο τέλος της γραμμής ανάθεσής της. Το πλήρες m-script φαίνεται παρακάτω.

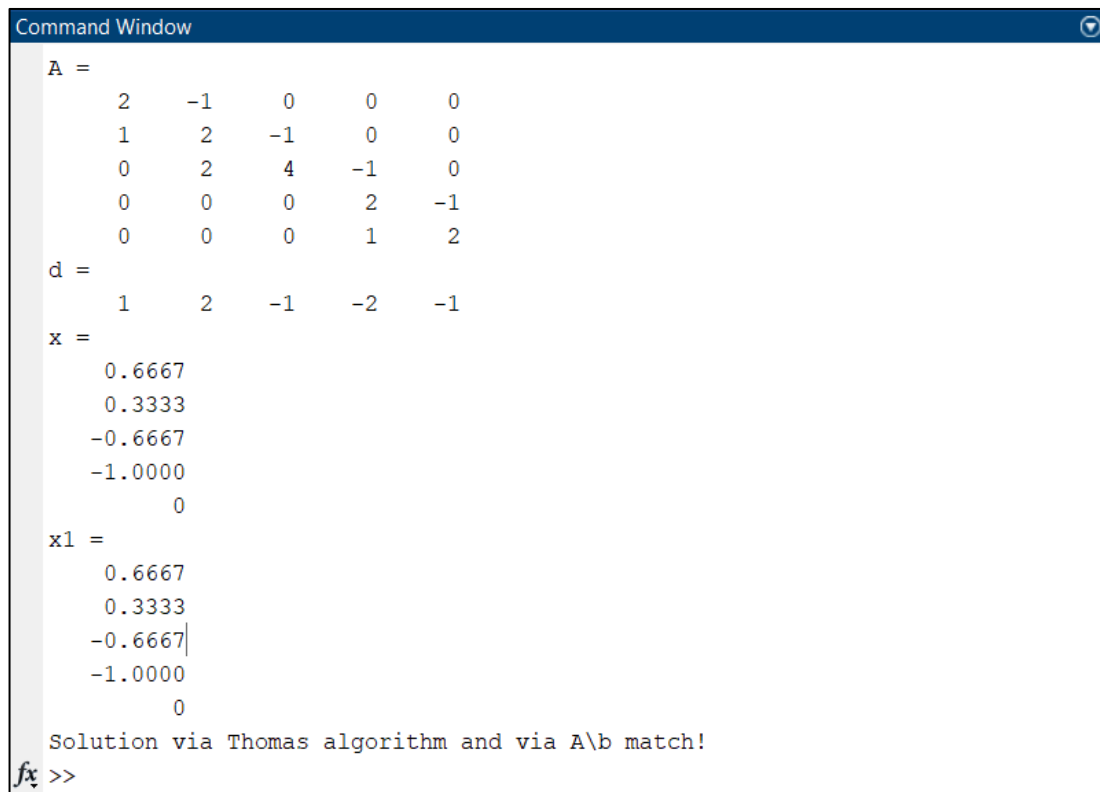
Προς έλεγχο της ορθότητας της λύσης με την μέθοδο Thomas, το δεδομένο σύστημα επιλύεται και με την βοήθεια του τελεστή $A \setminus d'$ της Matlab, όπου ο πίνακας A του συστήματος «αντιστρέφεται» και πολλαπλασιάζεται με τον *ανάστροφο* d' του διανύσματος των σταθερών d (η χρήση του αναστρόφου επιβάλλεται, καθώς το διάνυσμα d ορίζεται ως γραμμή). Η προκύπτουσα λύση με αυτή την μέθοδο αποθηκεύεται στην μεταβλητή $x1$. Στην συνέχεια πραγματοποιείται έλεγχος του κατά πόσο οι δύο λύσεις, x και $x1$, ταυτίζονται ή όχι, εντός της ακρίβειας της Matlab (χρησιμοποιείται η παράμετρος του λειτουργικού συστήματος της Matlab, ϵ). Σε περίπτωση όπου η απόλυτη τιμή της διαφοράς των δύο λύσεων είναι μικρότερη της μεταβλητής ϵ , εκτυπώνεται στο τερματικό κατάλληλο μήνυμα.

MATLAB SCRIPT M-FILE (ThomasAlgorithmEx1_1_Script.m):

```
clear
clc

% --- CFD Exercise 1.1 ---
% Solution script (using ThomasAlgorithm.m)
%
% --- Definition of matrix A ---
A = [2 -1 0 0 0;
     1 2 -1 0 0;
     0 2 4 -1 0;
     0 0 0 2 -1;
     0 0 0 1 2]

%
% --- Definition of constant vector d ---
d = [1 2 -1 -2 -1]
% Note: It is ok to define d as a row. The function
% in "ThomasAlgorithm.m" will take care of it.
%
% --- Apply Thomas algorithm and get solution ---
x = ThomasAlgorithm(A,d)
%
% --- Check validity of solution ---
x1 = A\d'
if(abs(x-x1)<eps)
    disp('Solution via Thomas algorithm and via A\b match!')
end
% End of script.
```



Command Window

```
A =
     2     -1      0      0      0
     1      2     -1      0      0
     0      2      4     -1      0
     0      0      0      2     -1
     0      0      0      1      2

d =
     1      2     -1     -2     -1

x =
     0.6667
     0.3333
    -0.6667
    -1.0000
         0

x1 =
     0.6667
     0.3333
    -0.6667
    -1.0000
         0

Solution via Thomas algorithm and via A\b match!
fx >>
```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του δεδομένου συστήματος και με τις δύο μεθόδους, όπως φαίνεται παραπάνω, καθώς και το μήνυμα ότι οι δύο λύσεις ταυτίζονται εντός της ακρίβειας της Matlab.

Βάσει των παραπάνω, συμπεραίνουμε ότι η λύση του δεδομένου συστήματος, όπως προέκυψε με την επίλυσή του κάνοντας χρήση του αλγορίθμου Thomas, είναι η εξής:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 0.6667 \\ 0.3333 \\ -0.6667 \\ -1.0000 \\ 0.0000 \end{bmatrix} \equiv \begin{bmatrix} 2/3 \\ 1/3 \\ -2/3 \\ -1 \\ 0 \end{bmatrix} \quad (1.2)$$

όπου στην τελευταία «ισότητα» εμφανίζεται η λύση του συστήματος, όπως προκύπτει «αναλυτικά» μέσω απαλοιφής Gauss (ή, εφαρμόζοντας τον αλγόριθμο Thomas δια χειρός). Είναι φανερό ότι η συνάρτηση που υλοποιήθηκε παραπάνω λειτουργεί σωστά, και αποδίδει ορθά αποτελέσματα. ■

Άσκηση 1.2

Υλοποιήστε στην γλώσσα προγραμματισμού Matlab την επίλυση του συστήματος της άσκησης Exercise set 7.3 : 1(c) (σελίδα 459), ήτοι το σύστημα:

$$\begin{aligned} 10x_1 + 5x_2 &= 6 \\ 5x_1 + 10x_2 - 4x_3 &= 25 \\ -4x_2 + 8x_3 - x_4 &= -11 \\ -x_3 + 5x_4 &= -11 \end{aligned} \Leftrightarrow \begin{bmatrix} 10 & 5 & 0 & 0 \\ 5 & 10 & -4 & 0 \\ 0 & -4 & 8 & -1 \\ 0 & 0 & -1 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 6 \\ 25 \\ -11 \\ -11 \end{bmatrix}$$

με τις μεθόδους:

1. Jacobi.
2. Gauss – Seidel.
3. Διαδοχικής χαλάρωσης με $\omega = 1.1$.

Μπορείτε να θεωρήσετε ως μία αρχική εκτίμηση της λύσης την $\mathbf{x}^{(0)} = \mathbf{0}$.

Λύση

Προς επίλυση του προβλήματος, μετασχηματίζουμε αρχικά το δεδομένο σύστημα σε μορφή πινάκων, όπως σημειώνεται και στην εκφώνηση, ήτοι στην μορφή:

$$\begin{bmatrix} 10 & 5 & 0 & 0 \\ 5 & 10 & -4 & 0 \\ 0 & -4 & 8 & -1 \\ 0 & 0 & -1 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 6 \\ 25 \\ -11 \\ -11 \end{bmatrix} \quad (1.3)$$

όπου γίνεται εμφανές το γεγονός ότι ο πίνακας του συστήματος είναι *διαγώνια κυρίαρχος*, όπως μπορεί να ελεγχθεί τετριμμένα (ο έλεγχος πραγματοποιείται και στο πρόγραμμα, ωστόσο είναι καλή πρακτική να γίνεται και «δια χειρός»). Ως *συνθήκη τερματισμού*, επιλέγεται για όλες τις μεθόδους η εξής:

$$\frac{\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_{\infty}}{\|\mathbf{x}^{(k+1)}\|_{\infty}} < \varepsilon = 10^{-5} \quad (1.4)$$

όπου $\|\cdot\|_{\infty}$ η νόρμα μεγίστου στοιχείου, και η παράμετρος τερματισμού (ή, ακρίβεια) επιλέγεται στο 5^ο σημαντικό ψηφίο ($\varepsilon = 10^{-5}$). Η ακρίβεια επιλέγεται τέτοια, ώστε στην προεπιλεγμένη επιλογή (format) της Matlab, η οποία εμφανίζει τα αποτελέσματα με μόνο 4 δεκαδικά ψηφία, να εμφανίζονται μόνον *εγγυημένα* ψηφία. Με την αντίστοιχη εντολή `format long` στο τερματικό θα εμφανιστούν και τα *μη εγγυημένα* ψηφία των αποτελεσμάτων που προκύπτουν από εφαρμογή της μεθόδου προς επίλυση του δεδομένου συστήματος.

1. Υλοποιείται σε γλώσσα Matlab ο αλγόριθμος Jacobi, ακολουθώντας την αλγοριθμική διαδικασία των Σημειώσεων του μαθήματος. Ο πηγαίος κώδικας γράφεται, και αποθηκεύεται, σε ένα αρχείο Matlab συνάρτησης (m-function file) με όνομα “JacobiMethod.m”, και παρατίθεται παρακάτω. Η συνάρτηση δέχεται ως παραμέτρους τον τετραγωνικό πίνακα, **A**, του συστήματος, το διάνυσμα σταθερών, **b**, την αρχική εκτίμηση, **x_initial**, καθώς και την επιθυμητή ακρίβεια, **epsilon**, και

επιστρέφει το διάνυσμα x_final , ήτοι την λύση του συστήματος υπό την επιλεγμένη συνθήκη τερματισμού, καθώς και τον αριθμό επαναλήψεων, $iter$, που απαιτήθηκαν προς επίτευξή της. Αρχικά, μετά τον ορισμό της διάστασης του συστήματος (μεταβλητή n), γίνεται έλεγχος της ορθότητας μορφής των δεδομένων εισόδου. Συγκεκριμένα, το διάνυσμα σταθερών, b , μετατρέπεται (αν δεν είναι ήδη) σε μορφή στήλης, και το ίδιο γίνεται για το διάνυσμα $x_initial$, ενώ αξιοποιώντας την εντολή $size(A)$, ελέγχεται εάν οι διαστάσεις των εμπλεκόμενων πινάκων είναι ίσες (οπότε το σύστημα είναι «τετραγωνικό»). Επιπλέον, ελέγχεται το κατά πόσο ο εισαγόμενος πίνακας A χαρακτηρίζεται από *διαγώνια υπεροχή*, μέσω κατάλληλου βρόγχου `for`, όπου υπολογίζεται ανά γραμμή το άθροισμα των απολύτων τιμών των στοιχείων του πίνακα του συστήματος (χωρίς συνεισφορά του εκάστοτε διαγώνιου στοιχείου), και ελέγχεται εάν πληρείται η απαιτούμενη συνθήκη διαγώνιας υπεροχής. Τέλος, ελέγχεται εάν ο πίνακας A του συστήματος περιλαμβάνει κάποιο μηδενικό διαγώνιο στοιχείο (περίπτωση κατά την οποία η μέθοδος Jacobi αποκλίνει). Σε περίπτωση που κάποια απαραίτητη συνθήκη για να είναι το πρόβλημα καλώς ορισμένο δεν πληρείται, το πρόγραμμα σταματά, και επιστρέφεται το αντίστοιχο μήνυμα σφάλματος στο `prompt window`, με την αντίστοιχη εντολή `error()`, με προφανή λειτουργία.

Μετά τον έλεγχο ορθότητας των δεδομένων εισόδου, η συνάρτηση υλοποιεί την μέθοδο Jacobi. Προς αυτό τον σκοπό, αρχικοποιούνται οι μεταβλητές εξόδου της συνάρτησης, καθώς και η «προσωρινές» (temporary) μεταβλητές για τον επαναληπτικό βρόγχο της μεθόδου. Συγκεκριμένα, αρχικοποιείται ένα «προσωρινό» διάνυσμα στήλη, x_old , στην τιμή του διανύσματος $x_initial$ της αρχικής εκτίμησης. Σκοπός του διανύσματος x_old είναι σε αυτό να αντιγράφονται κάθε φορά τα αποτελέσματα της «περασμένης» επανάληψης της μεθόδου. Αρχικοποιείται επίσης το διάνυσμα εξόδου, x_final , το οποίο, για κάθε επανάληψη, αποτελεί το «τελικό» αποτέλεσμα της μεθόδου (η αρχικοποίηση του x_final είναι αυθαίρετη, εφόσον δύναται να αλλάξει ήδη από την πρώτη επανάληψη). Επιπλέον, αρχικοποιείται ο αριθμός επαναλήψεων στην τιμή 0, καθώς και η παράμετρος τερματισμού, `terminationRatio`, η οποία αρχικά τίθεται αυθαίρετα σε μία τιμή μεγαλύτερη της εισαγόμενης ακρίβειας, `epsilon`. Μετά την αρχικοποίηση των μεταβλητών εξόδου και των «προσωρινών» μεταβλητών, υλοποιείται ο κύριος βρόγχος `while` της μεθόδου. Συγκεκριμένα, όσο η λογική συνθήκη του βρόγχου `while` παραμένει αληθής, το πρόγραμμα υπολογίζει, κατά συνιστώσα, την λύση της «τρέχουσας» επανάληψης, x_final , με δεδομένη την λύση της «προηγούμενης» επανάληψης, x_old , βάσει της σχέσης:

$$\text{Μέθοδος Jacobi : } x_i^{(k+1)} = \frac{1}{a_{ii}} \left[b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right], i = 1, 2, \dots, n \quad (1.5)$$

η οποία ορίζει και την μέθοδο Jacobi. Το άθροισμα υπολογίζεται κάθε φορά με κατάλληλο βρόγχο `for`, σε συνδυασμό με κατάλληλη «προσωρινή» μεταβλητή, s . Πριν τον τερματισμό κάθε «εξωτερικού» βρόγχου `while`, ο αριθμός επαναλήψεων αυξάνεται κατά 1, η παράμετρος τερματισμού, `terminationRatio`, τίθεται στην νέα τιμή (από την σχέση (1.4) παραπάνω), και η «τρέχουσα» λύση, x_final , ανατίθεται

στην λύση της «προηγούμενης» επανάληψης, x_{old} . Ο πλήρης πηγαίος κώδικας που υλοποιεί τα παραπάνω παρουσιάζεται παρακάτω.

MATLAB FUNCTION M-FILE (JacobiMethod.m):

```
function [x_final, iter] = JacobiMethod(A, b, x_initial, epsilon)
% Function JacobiMethod.m
%-----
% Jacobi's iterative method for linear system A*x = b.
% Input : - square matrix A of coefficients      (n x n).
%          - const. vector b                    (n x 1).
%          - initial guess x_initial            (n x 1).
%          - termination parameter epsilon      (1 x 1).
%          (use of "weighted" infinity norm)
% Output: - vector x_final = sol. of (Ax=b)      (n x 1).
%          - integer iter = no. of iterations    (1 x 1).
%-----

% --- System dimension definition ---
n = length(b);

% --- Input validity ---
% Ensure epsilon is positive:
if(epsilon <= 0)
    error('Termination parameter epsilon must be positive.');
```

```
x_final = x_old;
iter = 0;
terminationRatio = 1 + epsilon;
% --- 2. Main iteration loop ---
while(terminationRatio >= epsilon)
    for i = 1:n
        s = 0;
        for j = 1:n
            if(j ~= i)
                s = s + A(i,j)*x_old(j);
            end
        end
        x_final(i) = (b(i) - s)/A(i,i);
    end
    iter = iter + 1;
    terminationRatio = norm(x_final-x_old, "inf")/norm(x_final,"inf");
    x_old = x_final;
end
end
```

Βάσει της παραπάνω συνάρτησης, η επίλυση του συστήματος γίνεται τετριμμένη, και παρόλο που μπορεί να γίνει απευθείας στο Command Window, παρουσιάζεται για εποπτικούς λόγους στο παρακάτω αρχείο (m-script file). Αρχικά ορίζεται ο πίνακας A του συστήματος, στην συνέχεια το διάνυσμα b των σταθερών (το οποίο, λόγω του εξονυχιστικού ελέγχου των δεδομένων εισόδου στην παραπάνω υλοποίηση της μεθόδου *Jacobi* δεν υπάρχει πρόβλημα να οριστεί ως γραμμή), η αρχική εκτίμηση $x_initial$ η οποία λαμβάνεται να είναι το μηδενικό διάνυσμα, και η επιλεγμένη ακρίβεια $epsilon$ στο 5^ο δεκαδικό ψηφίο. Έπειτα η λύση του συστήματος, με εφαρμογή της συνάρτησης “*JacobiMethod(A,b,x_initial,epsilon)*”, αποθηκεύεται σε ζεύγος μεταβλητών $[x, iter]$, το οποίο εκτυπώνεται στο τερματικό αφού δεν τοποθετείται “;” στο τέλος της γραμμής ανάθεσής του. Το πλήρες m-script φαίνεται παρακάτω.

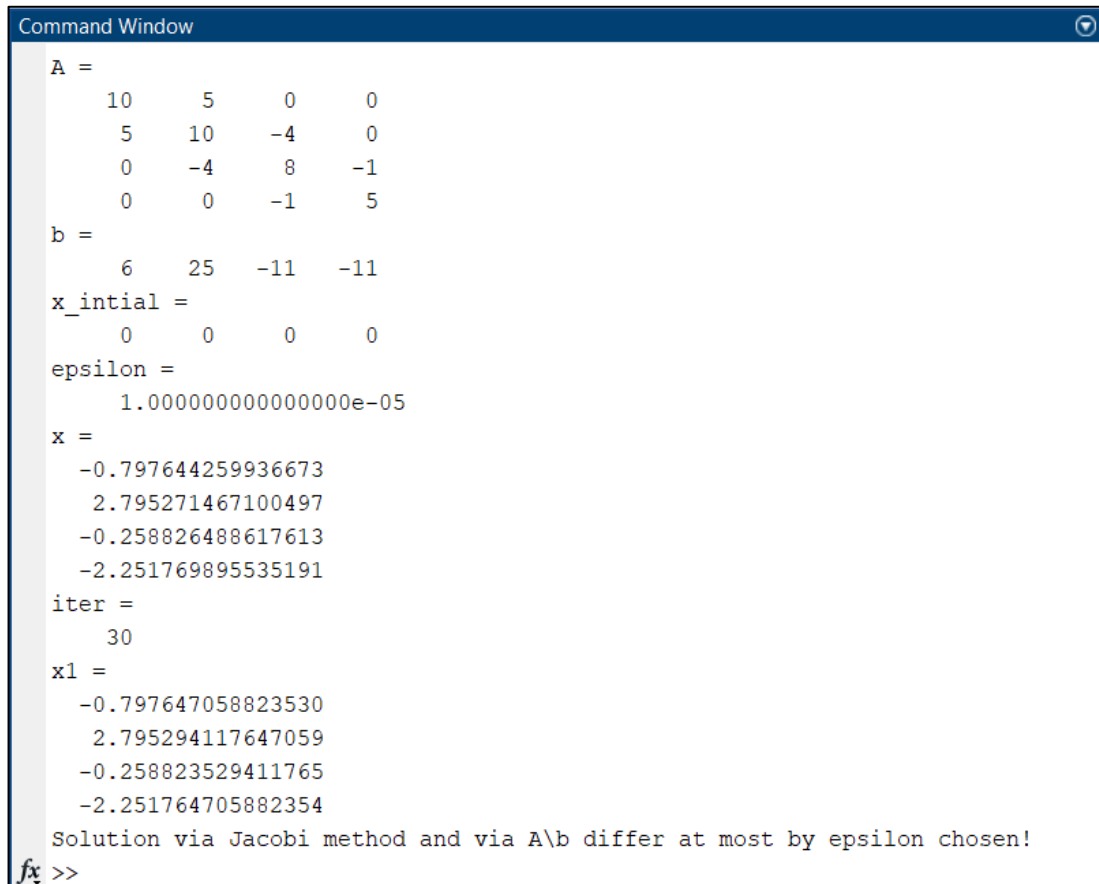
Προς έλεγχο της ορθότητας της λύσης με την μέθοδο *Jacobi*, το δεδομένο σύστημα επιλύεται και με την βοήθεια του τελεστή $A \setminus b'$ της Matlab, όπου ο πίνακας A του συστήματος «αντιστρέφεται» και πολλαπλασιάζεται με τον *αναστρόφο* b' του διανύσματος των σταθερών b (η χρήση του αναστρόφου επιβάλλεται, καθώς το διάνυσμα b ορίζεται ως γραμμή). Η προκύπτουσα λύση με αυτή την μέθοδο αποθηκεύεται στην μεταβλητή $x1$. Στην συνέχεια πραγματοποιείται έλεγχος του κατά πόσο οι δύο λύσεις, x και $x1$, ταυτίζονται ή όχι, εντός της ακρίβειας $epsilon$. Σε περίπτωση όπου η απόλυτη τιμή της διαφοράς των δύο λύσεων είναι μικρότερη της μεταβλητής $epsilon$, εκτυπώνεται στο τερματικό κατάλληλο μήνυμα.

MATLAB SCRIPT M-FILE (JacobiMethodEx1_2_Script.m):

```
clear
clc

% --- CFD Exercise 1.2 ---
% Solution script (using JacobiMethod.m)
%
% --- Definition of matrix A ---
A = [10 5 0 0;
```

```
5 10 -4 0;  
0 -4 8 -1;  
0 0 -1 5]  
  
%  
% --- Definition of constant vector b ---  
b = [6 25 -11 -11]  
% Note: It is ok to define b as a row. The function  
% in "JacobiMethod.m" will take care of it.  
%  
% --- Definition of x_initial ---  
x_initial = [0 0 0 0]  
%  
% --- Definition of accuracy epsilon ---  
epsilon = 1e-5  
% --- Apply Jacobi method and get solution ---  
[x,iter] = JacobiMethod(A,b,x_initial,epsilon)  
%  
% --- Check validity of solution ---  
x1 = A\b'  
if(abs(x-x1)<epsilon)  
    disp(['Solution via Jacobi method and via A\b match ' ...  
        'within epsilon chosen!'])  
else  
    disp(['Solution via Jacobi method and via A\b differ ' ...  
        'at most by epsilon chosen!'])  
end  
% End of script.
```



```
Command Window  
A =  
    10     5     0     0  
     5    10    -4     0  
     0    -4     8    -1  
     0     0    -1     5  
  
b =  
     6    25   -11   -11  
  
x_initial =  
     0     0     0     0  
  
epsilon =  
    1.000000000000000e-05  
  
x =  
   -0.797644259936673  
    2.795271467100497  
   -0.258826488617613  
   -2.251769895535191  
  
iter =  
    30  
  
x1 =  
   -0.797647058823530  
    2.795294117647059  
   -0.258823529411765  
   -2.251764705882354  
  
Solution via Jacobi method and via A\b differ at most by epsilon chosen!  
fx >>
```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του δεδομένου συστήματος και με τις δύο μεθόδους, όπως φαίνεται παραπάνω (σε μορφή εκτύπωσης `format long; format compact`), καθώς και το μήνυμα ότι οι δύο λύσεις δεν ταυτίζονται εντός της ακρίβειας `epsilon`, όπως άλλωστε αναμένεται, καθώς η επαναληπτική μέθοδος Jacobi έδωσε την υπό συζήτηση λύση υπό την ακρίβεια αυτή (σε αντίθεση με την «εσωτερική» μέθοδο `A\b` της Matlab η οποία παρέχει πολύ καλύτερη ακρίβεια). Λαμβάνουμε επίσης τον αριθμό επαναλήψεων που απαιτήθηκαν από την εν λόγω μέθοδο για επίτευξη της απαιτούμενης ακρίβειας της λύσης του δεδομένου συστήματος.

Βάσει των παραπάνω, συμπεραίνουμε ότι η λύση του δεδομένου συστήματος, όπως προέκυψε με την επίλυσή του κάνοντας χρήση της μεθόδου Jacobi, είναι η εξής:

$$\mathbf{x}_{\text{Jacobi}} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -0.7976 \\ 2.7953 \\ -0.2588 \\ -2.2518 \end{bmatrix}, \quad N_{\text{iterations}}^{\text{Jacobi}} = 30, \quad \varepsilon = 10^{-5} \quad (1.6)$$

όπου τα αποτελέσματα αναγράφονται με τέσσερα *εγγυημένα* δεκαδικά ψηφία (όπως λαμβάνονται υπό την εντολή `format default` στο Command Window του Matlab), όπως επιβάλλει η επιλεγόμενη ακρίβεια. Επιπλέον, σημειώνεται στο παραπάνω αποτέλεσμα και ο αριθμός των επαναλήψεων που απαιτήθηκαν ώστε να ληφθεί η συγκεκριμένη ακρίβεια στην λύση του δεδομένου συστήματος.

2. Η διαδικασία υλοποίησης της μεθόδου Gauss – Seidel για την επίλυση του δεδομένου συστήματος είναι εντελώς όμοια με την παραπάνω, με εξαίρεση το γεγονός ότι στο κύριο βρόγχο επανάληψης του αλγορίθμου, πρέπει να πραγματοποιηθεί κάποια τροποποίηση, ώστε να υλοποιούνται οι διαφορές μεταξύ της μεθόδου Jacobi και της μεθόδου Gauss – Seidel.

Υλοποιείται σε γλώσσα Matlab ο αλγόριθμος Gauss – Seidel, ακολουθώντας την αλγοριθμική διαδικασία των Σημειώσεων του μαθήματος. Ο πηγαίος κώδικας γράφεται, και αποθηκεύεται, σε ένα αρχείο Matlab συνάρτησης (m-function file) με όνομα “GaussSeidelMethod.m”, και παρατίθεται παρακάτω. Η συνάρτηση δέχεται ως παραμέτρους τον τετραγωνικό πίνακα, **A**, του συστήματος, το διάνυσμα σταθερών, **b**, την αρχική εκτίμηση, `x_initial`, καθώς και την επιθυμητή ακρίβεια, `epsilon`, και επιστρέφει το διάνυσμα `x_final`, ήτοι την λύση του συστήματος υπό την επιλεγμένη συνθήκη τερματισμού, καθώς και τον αριθμό επαναλήψεων, `iter`, που απαιτήθηκαν προς επίτευξή της. Ο έλεγχος ορθότητας των δεδομένων εισόδου γίνεται ακριβώς με τον ίδιο τρόπο, όπως για την μέθοδο Jacobi, που παρουσιάστηκε παραπάνω.

Μετά τον έλεγχο ορθότητας των δεδομένων εισόδου, η συνάρτηση υλοποιεί την μέθοδο Gauss - Seidel. Προς αυτό τον σκοπό, αρχικοποιούνται οι μεταβλητές εξόδου της συνάρτησης, καθώς και η «προσωρινές» (temporary) μεταβλητές για τον επαναληπτικό βρόγχο της μεθόδου. Συγκεκριμένα, αρχικοποιείται ένα «προσωρινό» διάνυσμα στήλη, `x_old`, στην τιμή του διανύσματος `x_initial` της αρχικής εκτίμησης. Σκοπός του διανύσματος `x_old` είναι σε αυτό να αντιγράφονται κάθε φορά τα αποτελέσματα της «περασμένης» επανάληψης της μεθόδου. Αρχικοποιείται επίσης

το διάνυσμα εξόδου, x_final , το οποίο, για κάθε επανάληψη, αποτελεί το «τελικό» αποτέλεσμα της μεθόδου (η αρχικοποίηση του x_final είναι αυθαίρετη, εφόσον δύναται να αλλάξει ήδη από την πρώτη επανάληψη). Επιπλέον, αρχικοποιείται ο αριθμός επαναλήψεων στην τιμή 0, καθώς και η παράμετρος τερματισμού, $terminationRatio$, η οποία αρχικά τίθεται αυθαίρετα σε μία τιμή μεγαλύτερη της εισαγόμενης ακρίβειας, $epsilon$. Μετά την αρχικοποίηση των μεταβλητών εξόδου και των «προσωρινών» μεταβλητών, υλοποιείται ο κύριος βρόγχος `while` της μεθόδου. Συγκεκριμένα, όσο η λογική συνθήκη του βρόγχου `while` παραμένει αληθής, το πρόγραμμα υπολογίζει, κατά συνιστώσα, την λύση της «τρέχουσας» επανάληψης, x_final , με δεδομένη την λύση της «προηγούμενης» επανάληψης, x_old , βάσει της σχέσης:

$$\text{Μέθοδος Gauss - Seidel : } x_i^{(k+1)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right], i = 1, \dots, n \quad (1.7)$$

η οποία ορίζει και την μέθοδο Gauss - Seidel. Το άθροισμα υπολογίζεται κάθε φορά με κατάλληλο βρόγχο `for`, σε συνδυασμό με κατάλληλη «προσωρινή» μεταβλητή, s , ενώ πλέον, αξιοποιούνται στο άθροισμα και οι ήδη γνωστές τιμές της «τρέχουσας» επανάληψης. Πριν τον τερματισμό κάθε «εξωτερικού» βρόγχου `while`, ο αριθμός επαναλήψεων αυξάνεται κατά 1, η παράμετρος τερματισμού, $terminationRatio$, τίθεται στην νέα τιμή (από την σχέση (1.4) παραπάνω), και η «τρέχουσα» λύση, x_final , ανατίθεται στην λύση της «προηγούμενης» επανάληψης, x_old . Ο πλήρης πηγαίος κώδικας που υλοποιεί τα παραπάνω παρουσιάζεται παρακάτω.

MATLAB FUNCTION M-FILE (GaussSeidelMethod.m):

```
function [x_final, iter] = GaussSeidelMethod(A, b, x_initial, epsilon)
% Function GaussSeidelMethod.m
%-----
% Gauss-Seidel iterative method for linear system A*x = b.
% Input : - square matrix A of coefficients    (n x n).
%          - const. vector b                  (n x 1).
%          - initial guess x_initial           (n x 1).
%          - termination parameter epsilon     (1 x 1).
%          (use of "weighted" infinity norm)
% Output: - vector x_final = sol. of (Ax=b)    (n x 1).
%          - integer iter = no. of iterations  (1 x 1).
%-----

% --- System dimension & solution definition ---
n = length(b);

% --- Input validity ---
% Ensure epsilon is positive:
if(epsilon <= 0)
    error('Termination parameter epsilon must be positive.');
```

```
if(m1 ~= m2)
    error('Matrix A is not square (n x n).');
elseif(m1 ~= n)
    error('Matrix A and vector b are incompatible.');
```

```
elseif(m1 ~= length(x_initial))
    error('Initial guess x_initial is incompatible with matrices A,b.');
```

```
end
% Ensure A is diagonally dominant:
Asumrow = zeros(n,1);
counter = 0;
for i = 1:n
    Asumrow(i) = sum(abs(A(i,:))) - abs(A(i,i));
    if(Asumrow(i) <= abs(A(i,i)))
        counter = counter + 1;
    end
end
if(counter ~= n)
    error('Matrix A is not diagonally dominant. Method may diverge.');
```

```
end
% Ensure A has nonzero diagonal elements:
if(any(diag(A) == 0))
    error('Matrix A has a zero as diagonal element. Method diverges');
```

```
end

% --- Gauss-Seidel's method implementation ---
% --- 1. Temporary and Input variables initialization ---
x_old = x_initial;
x_final = x_old;
iter = 0;
terminationRatio = 1 + epsilon;
% --- 2. Main iteration loop ---
while(terminationRatio >= epsilon)
    for i = 1:n
        s = 0;
        for j = 1:n
            if(j < i)
                s = s + A(i,j)*x_final(j);
            elseif(j > i)
                s = s + A(i,j)*x_old(j);
            end
        end
        x_final(i) = (b(i) - s)/A(i,i);
    end
    iter = iter + 1;
    terminationRatio = norm(x_final-x_old, "inf")/norm(x_final,"inf");
    x_old = x_final;
end
end
```

Βάσει της παραπάνω συνάρτησης, η επίλυση του συστήματος γίνεται τετριμμένη, και παρόλο που μπορεί να γίνει απευθείας στο Command Window, παρουσιάζεται για εποπτικούς λόγους στο παρακάτω αρχείο (m-script file), η υλοποίηση του οποίου ακολουθεί την λογική υλοποίησης του αντίστοιχου m-script αρχείου της μεθόδου Jacobi παραπάνω. Προς έλεγχο της ορθότητας της λύσης με την μέθοδο Gauss – Seidel, το δεδομένο σύστημα επιλύεται τόσο με την βοήθεια της μεθόδου Jacobi, όσο και του τελεστή $A \backslash b$ της Matlab. Οι προκύπτουσες λύσεις μέσω των μεθόδων Jacobi και

Gauss – Seidel αποθηκεύονται στα ζεύγη μεταβλητών $[xJ, iterJ]$ και $[xGS, iterGS]$, αντίστοιχα, ενώ η λύση από την μέθοδο $A \setminus b$ αποθηκεύεται στην μεταβλητή $x1$. Στην συνέχεια πραγματοποιείται έλεγχος του κατά πόσο οι λύσεις από τις δύο μεθόδους ταυτίζονται ή όχι με αυτή που προκύπτει από τον τελεστή $A \setminus b$, εντός της ακρίβειας ϵ , και επιστρέφεται κατάλληλο μήνυμα (προφανώς, το αποτέλεσμα που αναμένεται είναι οι λύσεις xJ και xGS να μην ταυτίζονται με την $x1$, λόγω της επιλεγόμενης εξ ορισμού ακρίβειας των δύο μεθόδων – σύγκριση και ανάλυση περί αυτού γίνεται παρακάτω).

MATLAB SCRIPT M-FILE (GaussSeidelMethodEx1_2_Script.m):

```
clear
clc

% --- CFD Exercise 1.2 ---
% Solution script (using JacobiMethod.m, GaussSeidelMethod.m)
%
% --- Definition of matrix A ---
A = [10 5 0 0;
     5 10 -4 0;
     0 -4 8 -1;
     0 0 -1 5]
%
% --- Definition of constant vector b ---
b = [6 25 -11 -11]
% Note: It is ok to define b as a row. The functions
% used will take care of it.
%
% --- Definition of x_initial ---
x_initial = [0 0 0 0]
%
% --- Definition of accuracy epsilon ---
epsilon = 1e-5
% --- Apply Jacobi method and get solution ---
[xJ, iterJ] = JacobiMethod(A, b, x_initial, epsilon)
%
% --- Apply Gauss-Seidel method and get solution ---
[xGS, iterGS] = GaussSeidelMethod(A, b, x_initial, epsilon)
%
% --- Check validity of solutions ---
x1 = A \ b
if(abs(xJ-x1)<epsilon)
    disp(['Solution via Jacobi method and via A\b match ' ...
        'within epsilon chosen!'])
else
    disp(['Solution via Jacobi method and via A\b differ ' ...
        'at most by epsilon chosen!'])
end
if(abs(xGS-x1)<epsilon)
    disp(['Solution via Gauss-Seidel method and via A\b match ' ...
        'within epsilon chosen!'])
else
    disp(['Solution via Gauss-Seidel method and via A\b differ ' ...
        'at most by epsilon chosen!'])
end
% End of script.
```

```

Command Window

A =
    10     5     0     0
     5    10    -4     0
     0    -4     8    -1
     0     0    -1     5

b =
     6    25   -11   -11

x_initial =
     0     0     0     0

epsilon =
    1.0000000000000000e-05

xJ =
   -0.797644259936673
    2.795271467100497
   -0.258826488617613
   -2.251769895535191

iterJ =
    30

xGS =
   -0.797631735940593
    2.795279975977254
   -0.258831005260938
   -2.251766201052187

iterGS =
    14

x1 =
   -0.797647058823530
    2.795294117647059
   -0.258823529411765
   -2.251764705882354

Solution via Jacobi method and via A\b differ at most by epsilon chosen!
Solution via Gauss-Seidel method and via A\b differ at most by epsilon chosen!

fx >>

```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του δεδομένου συστήματος και με τις δύο μεθόδους, όπως φαίνεται παραπάνω (σε μορφή εκτύπωσης `format long`; `format compact`), καθώς και το μήνυμα ότι οι δύο λύσεις δεν ταυτίζονται με αυτή που προέκυψε από τον τελεστή `A\b`, εντός της ακρίβειας `epsilon`, όπως άλλωστε αναμένεται, καθώς οι επαναληπτικές μέθοδοι ορίζουν τις υπό συζήτηση λύσεις υπό την ακρίβεια αυτή (σε αντίθεση με την «εσωτερική» μέθοδο `A\b` της Matlab η οποία παρέχει πολύ καλύτερη ακρίβεια). Λαμβάνουμε επίσης τον αριθμό επαναλήψεων που απαιτήθηκαν από την εν λόγω μέθοδο για επίτευξη της απαιτούμενης ακρίβειας της λύσης του δεδομένου συστήματος, και παρατηρούμε ότι η μέθοδος Gauss – Seidel απαιτεί εμφανώς λιγότερες επαναλήψεις από την μέθοδο Jacobi ώστε να επιτευχθεί η ζητούμενη ακρίβεια. Ο λόγος για τον οποίο η μέθοδος Gauss – Seidel υπερτερεί, ως προς την ταχύτητα σύγκλισης, σε σχέση με την μέθοδο Jacobi, αν και αποδεικνύεται με μεθόδους αριθμητικής ανάλυσης, «διαισθητικά» οφείλεται στο γεγονός ότι η πρώτη μέθοδος αξιοποιεί σε κάθε επανάληψη όλες τις ήδη γνωστές «τρέχουσες» λύσεις για τις συνιστώσες του ζητούμενου διανύσματος της λύσης. Αντίθετα, η μέθοδος Jacobi υπολογίζει τις τιμές

των συνιστωσών της «τρέχουσας» επανάληψης, αξιοποιώντας *μόνον* τις τιμές των συνιστωσών της «προηγούμενης» επανάληψης (παρόλο που κάποιες από αυτές έχουν ήδη «ενημερωθεί»).

Βάσει των παραπάνω, συμπεραίνουμε ότι η λύση του δεδομένου συστήματος, όπως προέκυψε με την επίλυσή του κάνοντας χρήση της μεθόδου Gauss – Seidel, είναι η:

$$\mathbf{x}_{\text{GaussSeidel}} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -0.7976 \\ 2.7953 \\ -0.2588 \\ -2.2518 \end{bmatrix}, \quad N_{\text{GaussSeidel}}^{\text{iterations}} = 14, \quad \varepsilon = 10^{-5} \quad (1.8)$$

όπου τα αποτελέσματα αναγράφονται με τέσσερα *εγγυημένα* δεκαδικά ψηφία (όπως λαμβάνονται υπό την εντολή `format default` στο Command Window του Matlab), όπως επιβάλλει η επιλεγόμενη ακρίβεια. Επιπλέον, σημειώνεται στο παραπάνω αποτέλεσμα και ο αριθμός των επαναλήψεων που απαιτήθηκαν ώστε να ληφθεί η συγκεκριμένη ακρίβεια στην λύση του δεδομένου συστήματος.

3. Ίδια βήματα με τα παραπάνω ακολουθούνται και για την επίλυση του δεδομένου συστήματος με μία Μέθοδο Διαδοχικής Υπερχαλάρωσης (Successive Over-Relaxation, SOR), για την οποία η επιθυμητή «παράμετρος χαλάρωσης» επιλέγεται να είναι $\omega = 1.1$. Η διαδικασία υλοποίησης μεθόδου SOR για την επίλυση του δεδομένου συστήματος είναι εντελώς όμοια με την παραπάνω, με εξαίρεση το γεγονός ότι στο κύριο βρόγχο επανάληψης του αλγορίθμου, πρέπει να πραγματοποιηθεί κάποια τροποποίηση, ώστε να υλοποιούνται οι διαφορές μεταξύ της μεθόδου Gauss – Seidel και της μεθόδου SOR, ήτοι να λαμβάνεται υπόψη και η παράμετρος χαλάρωσης ω .

Υλοποιείται σε γλώσσα Matlab ο αλγόριθμος SOR, ακολουθώντας την αλγοριθμική διαδικασία των Σημειώσεων του μαθήματος. Ο πηγαίος κώδικας γράφεται, και αποθηκεύεται, σε ένα αρχείο Matlab συνάρτησης (m-function file) με όνομα “`SORMethod.m`”, και παρατίθεται παρακάτω. Η συνάρτηση δέχεται ως παραμέτρους τον τετραγωνικό πίνακα, A , του συστήματος, το διάνυσμα σταθερών, b , την αρχική εκτίμηση, x_{initial} , την επιθυμητή ακρίβεια, epsilon , καθώς και την παράμετρο χαλάρωσης, ω , και επιστρέφει το διάνυσμα x_{final} , ήτοι την λύση του συστήματος υπό την επιλεγμένη συνθήκη τερματισμού, καθώς και τον αριθμό επαναλήψεων, iter , που απαιτήθηκαν προς επίτευξή της. Η παράμετρος χαλάρωσης ω ελέγχεται, ώστε να βρίσκεται εντός του διαστήματος $(0,2)$, απαραίτητη προϋπόθεση ώστε η μέθοδος SOR να συγκλίνει. Κατά τα άλλα, ο έλεγχος ορθότητας των δεδομένων εισόδου γίνεται ακριβώς με τον ίδιο τρόπο, όπως για τις μεθόδους Jacobi και Gauss - Seidel, που παρουσιάστηκαν παραπάνω.

Μετά τον έλεγχο ορθότητας των δεδομένων εισόδου, η συνάρτηση υλοποιεί την μέθοδο SOR. Προς αυτό τον σκοπό, αρχικοποιούνται οι μεταβλητές εξόδου της συνάρτησης, καθώς και η «προσωρινές» (temporary) μεταβλητές για τον επαναληπτικό βρόγχο της μεθόδου. Συγκεκριμένα, αρχικοποιείται ένα «προσωρινό» διάνυσμα στήλη, x_{old} , στην τιμή του διανύσματος x_{initial} της αρχικής εκτίμησης. Σκοπός του διανύσματος x_{old} είναι σε αυτό να αντιγράφονται κάθε φορά τα αποτελέσματα της «περασμένης» επανάληψης της μεθόδου. Αρχικοποιείται επίσης το διάνυσμα εξόδου, x_{final} , το οποίο, για κάθε επανάληψη, αποτελεί το «τελικό» αποτέλεσμα της

μεθόδου (η αρχικοποίηση του x_final είναι αυθαίρετη, εφόσον δύναται να αλλάξει ήδη από την πρώτη επανάληψη). Επιπλέον, αρχικοποιείται ο αριθμός επαναλήψεων στην τιμή 0, καθώς και η παράμετρος τερματισμού, `terminationRatio`, η οποία αρχικά τίθεται αυθαίρετα σε μία τιμή μεγαλύτερη της εισαγόμενης ακρίβειας, `epsilon`. Μετά την αρχικοποίηση των μεταβλητών εξόδου και των «προσωρινών» μεταβλητών, υλοποιείται ο κύριος βρόγχος `while` της μεθόδου. Συγκεκριμένα, όσο η λογική συνθήκη του βρόγχου `while` παραμένει αληθής, το πρόγραμμα υπολογίζει, κατά συνιστώσα, την λύση της «τρέχουσας» επανάληψης, x_final , με δεδομένη την λύση της «προηγούμενης» επανάληψης, x_old , βάσει της σχέσης:

$$\text{Μέθοδος SOR : } x_i^{(k+1)} = \frac{\omega}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right] + (1 - \omega) x_i^{(k)}, i = 1, \dots, n \quad (1.9)$$

η οποία ορίζει και την εν λόγω μέθοδο. Το άθροισμα υπολογίζεται κάθε φορά με κατάλληλο βρόγχο `for`, σε συνδυασμό με κατάλληλη «προσωρινή» μεταβλητή, s , ενώ αξιοποιούνται στο άθροισμα και οι ήδη γνωστές τιμές της «τρέχουσας» επανάληψης. Πριν τον τερματισμό κάθε «εξωτερικού» βρόγχου `while`, ο αριθμός επαναλήψεων αυξάνεται κατά 1, η παράμετρος τερματισμού, `terminationRatio`, τίθεται στην νέα τιμή (από την σχέση (1.4) παραπάνω), και η «τρέχουσα» λύση, x_final , ανατίθεται στην λύση της «προηγούμενης» επανάληψης, x_old . Ο πλήρης πηγαίος κώδικας που υλοποιεί τα παραπάνω παρουσιάζεται παρακάτω.

MATLAB FUNCTION M-FILE (SORMethod.m):

```
function [x_final, iter] = SORMethod(A, b, x_initial, epsilon, omega)
% Function SORMethod.m
%-----
% SOR iterative method for linear system A*x = b.
% Input : - square matrix A of coefficients      (n x n).
%          - const. vector b                    (n x 1).
%          - initial guess x_initial            (n x 1).
%          - termination parameter epsilon      (1 x 1).
%          (use of "weighted" infinity norm)
%          - relaxation parameter omega         (1 x 1).
% Output: - vector x_final = sol. of (Ax=b)     (n x 1).
%          - integer iter = no. of iterations   (1 x 1).
%-----

% --- System dimension & solution definition ---
n = length(b);

% --- Input validity ---
% Ensure omega is between 0 and 2:
if(omega <= 0 || omega >=2)
    error('Relaxation parameter omega must be in interval (0,2).');
end
% Ensure epsilon is positive:
if(epsilon <= 0)
    error('Termination parameter epsilon must be positive.');
```

```
% Ensure A and b are properly sized :
[m1,m2]=size(A);
if(m1 ~= m2)
    error('Matrix A is not square (n x n).');
elseif(m1 ~= n)
    error('Matrix A and vector b are incompatible. ');
elseif(m1 ~= length(x_initial))
    error('Initial guess x_initial is incompatible with matrices A,b. ');
end
% Ensure A is diagonally dominant:
Asumrow = zeros(n,1);
counter = 0;
for i = 1:n
    Asumrow(i) = sum(abs(A(i,:))) - abs(A(i,i));
    if(Asumrow(i) <= abs(A(i,i)))
        counter = counter + 1;
    end
end
if(counter ~= n)
    error('Matrix A is not diagonally dominant. Method may diverge. ');
end
% Ensure A has nonzero diagonal elements:
if(any(diag(A) == 0))
    error('Matrix A has a zero as diagonal element. Method diverges');
end

% --- SOR method implementation ---
% --- 1. Temporary and Input variables initialization ---
x_old = x_initial;
x_final = x_old;
iter = 0;
terminationRatio = 1 + epsilon;
% --- 2. Main iteration loop ---
while(terminationRatio >= epsilon)
    for i = 1:n
        s = 0;
        for j = 1:n
            if(j < i)
                s = s + A(i,j)*x_final(j);
            elseif(j > i)
                s = s + A(i,j)*x_old(j);
            end
        end
        x_final(i) = omega*(b(i) - s)/A(i,i) + (1-omega)*x_old(i);
    end
    iter = iter + 1;
    terminationRatio = norm(x_final-x_old, "inf") / norm(x_final, "inf");
    x_old = x_final;
end
end
```

Βάσει της παραπάνω συνάρτησης, η επίλυση του συστήματος γίνεται τετριμμένη, και παρόλο που μπορεί να γίνει απευθείας στο Command Window, παρουσιάζεται για εποπτικούς λόγους στο παρακάτω αρχείο (m-script file), η υλοποίηση του οποίου ακολουθεί την λογική υλοποίησης του αντίστοιχου m-script αρχείου της μεθόδου Gauss – Seidel παραπάνω. Προς έλεγχο της ορθότητας της λύσης με την μέθοδο SOR,

το δεδομένο σύστημα επιλύεται τόσο με την βοήθεια των μεθόδων Jacobi και Gauss - Seidel, όσο και του τελεστή $A \setminus b$ της Matlab. Οι προκύπτουσες λύσεις μέσω των μεθόδων SOR, Jacobi και Gauss – Seidel αποθηκεύονται στα ζεύγη μεταβλητών $[x_{SOR}, iter_{SOR}]$, $[x_J, iter_J]$ και $[x_{GS}, iter_{GS}]$, αντίστοιχα, ενώ η λύση από την μέθοδο $A \setminus b$ αποθηκεύεται στην μεταβλητή $x1$. Στην συνέχεια πραγματοποιείται έλεγχος του κατά πόσο οι λύσεις από τις τρεις μεθόδους ταυτίζονται ή όχι με αυτή που προκύπτει από τον τελεστή $A \setminus b$, εντός της ακρίβειας ϵ , και επιστρέφεται κατάλληλο μήνυμα.

MATLAB SCRIPT M-FILE (SORMethodEx1_2_Script.m):

```
clear
clc

% --- CFD Exercise 1.2 ---
% Solution script (using JacobiMethod.m, GaussSeidelMethod.m,
%                                     SORMethod.m)
%
% --- Definition of matrix A ---
A = [10 5 0 0;
     5 10 -4 0;
     0 -4 8 -1;
     0 0 -1 5]
%
% --- Definition of constant vector b ---
b = [6 25 -11 -11]
% Note: It is ok to define b as a row. The functions
% used will take care of it.
%
% --- Definition of x_initial ---
x_initial = [0 0 0 0]
%
% --- Definition of accuracy epsilon ---
epsilon = 1e-5
% --- Apply Jacobi method and get solution ---
[xJ,iterJ] = JacobiMethod(A,b,x_initial,epsilon)
%
% --- Apply Gauss-Seidel method and get solution ---
[xGS,iterGS] = GaussSeidelMethod(A,b,x_initial,epsilon)
%
% --- Apply SOR method (omega=1.1) and get solution ---
[xSOR,iterSOR] = SORMethod(A,b,x_initial,epsilon,1.1)
%
% --- Check validity of solutions ---
x1 = A\b'
if(abs(xJ-x1)<epsilon)
    disp(['Solution via Jacobi method and via A\b match ' ...
         'within epsilon chosen!'])
else
    disp(['Solution via Jacobi method and via A\b differ ' ...
         'at most by epsilon chosen!'])
end
if(abs(xGS-x1)<epsilon)
    disp(['Solution via Gauss-Seidel method and via A\b match ' ...
         'within epsilon chosen!'])
else
    disp(['Solution via Gauss-Seidel method and via A\b differ ' ...
```

```
        'at most by epsilon chosen!'])
end
if(abs(xSOR-x1)<epsilon)
    disp(['Solution via SOR method and via A\b match ' ...
        'within epsilon chosen!'])
else
    disp(['Solution via SOR method and via A\b differ ' ...
        'at most by epsilon chosen!'])
end
% End of script.
```

```
Command Window

      6      25     -11     -11
x_initial =
      0      0      0      0
epsilon =
      1.0000000000000000e-05
xJ =
    -0.797644259936673
     2.795271467100497
    -0.258826488617613
    -2.251769895535191
iterJ =
      30
xGS =
    -0.797631735940593
     2.795279975977254
    -0.258831005260938
    -2.251766201052187
iterGS =
      14
xSOR =
    -0.797653415344579
     2.795299174211020
    -0.258821269676919
    -2.251764346922075
iterSOR =
       8
x1 =
    -0.797647058823530
     2.795294117647059
    -0.258823529411765
    -2.251764705882354
Solution via Jacobi method and via A\b differ at most by epsilon chosen!
Solution via Gauss-Seidel method and via A\b differ at most by epsilon chosen!
Solution via SOR method and via A\b match within epsilon chosen!
fx >>
```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του δεδομένου συστήματος και με τις τρεις μεθόδους, όπως φαίνεται παραπάνω (σε μορφή εκτύπωσης `format long`; `format compact`), καθώς και το μήνυμα ότι οι δύο λύσεις, μέσω Jacobi και Gauss – Seidel, δεν ταυτίζονται με αυτή που προέκυψε από τον τελεστή `A\b`, εντός της ακρίβειας `epsilon`, όπως άλλωστε αναμένεται, καθώς οι επαναληπτικές μέθοδοι ορίζουν τις υπό συζήτηση λύσεις υπό την ακρίβεια αυτή (σε αντίθεση με την «εσωτερική» μέθοδο `A\b` της Matlab η οποία παρέχει πολύ

καλύτερη ακρίβεια). Ωστόσο, παρατηρούμε ότι η μέθοδος SOR, για την επιλεγόμενη τιμή παραμέτρου χαλάρωσης, δίνει λύση που *συμφωνεί* με την λύση που προκύπτει από τον τελεστή $A \setminus b'$, εντός της ακρίβειας ϵ , δηλαδή η μέθοδος διαδοχικής χαλάρωσης αποδίδει λύση που «τείνει να ξεπεράσει» την επιλεγόμενη παράμετρο ακρίβειας ϵ . Το γεγονός αυτό αναδεικνύει τόσο την ποιότητα της ίδιας της μεθόδου, και οφείλεται, μεταξύ άλλων, και την «ποιοτική» επιλογή της παραμέτρου χαλάρωσης ω .

Λαμβάνουμε επίσης τον αριθμό επαναλήψεων που απαιτήθηκαν από την εν λόγω μέθοδο για επίτευξη της απαιτούμενης ακρίβειας της λύσης του δεδομένου συστήματος, και παρατηρούμε ότι *η μέθοδος SOR απαιτεί εμφανώς τις λιγότερες επαναλήψεις τόσο από την μέθοδο Jacobi, όσο και από την μέθοδο Gauss – Seidel, ώστε να επιτευχθεί η ζητούμενη ακρίβεια*. Ο λόγος για τον οποίο η μέθοδος SOR υπερτερεί, ως προς την ταχύτητα σύγκλισης, σε σχέση με την μέθοδο Jacobi και την μέθοδο Gauss – Seidel, αν και αποδεικνύεται με μεθόδους αριθμητικής ανάλυσης, «διδασκαστικά» οφείλεται στο γεγονός ότι η μέθοδος χαλάρωσης «μετριάξει» την συνεισφορά μεταξύ «προηγούμενων» και «τρέχοντων» τιμών στην «τρέχουσα» συνιστώσα της λύσης. Αντίθετα, η μέθοδος Jacobi υπολογίζει τις τιμές των συνιστωσών της «τρέχουσας» επανάληψης, αξιοποιώντας *μόνον* τις τιμές των συνιστωσών της «προηγούμενης» επανάληψης (παρόλο που κάποιες από αυτές έχουν ήδη «ενημερωθεί»), ενώ η μέθοδος Gauss – Seidel αποδίδει εξ ολοκλήρου το «βάρος» στις «ενημερωμένες» τιμές.

Βάσει των παραπάνω, συμπεραίνουμε ότι η λύση του δεδομένου συστήματος, όπως προέκυψε με την επίλυσή του κάνοντας χρήση Μεθόδου Διαδοχικής Χαλάρωσης (SOR), με παράμετρο $\omega = 1.1$, είναι η εξής:

$$\mathbf{x}_{\text{SOR}} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -0.7976 \\ 2.7953 \\ -0.2588 \\ -2.2518 \end{bmatrix}, \quad N_{\text{iterations}}^{\text{SOR}} = 8, \quad \epsilon = 10^{-5} \quad (1.10)$$

όπου τα αποτελέσματα αναγράφονται με τέσσερα *εγγυημένα* δεκαδικά ψηφία (όπως λαμβάνονται υπό την εντολή `format default` στο Command Window του Matlab), όπως επιβάλλει η επιλεγόμενη ακρίβεια. Επιπλέον, σημειώνεται στο παραπάνω αποτέλεσμα και ο αριθμός των επαναλήψεων που απαιτήθηκαν ώστε να ληφθεί η συγκεκριμένη ακρίβεια στην λύση του δεδομένου συστήματος.

Συνοψίζοντας, όλες οι μέθοδοι που αξιοποιήθηκαν για την επίλυση του δεδομένου συστήματος αποδίδουν, εντός της επιλεγμένης ακρίβειας, και σύμφωνα με το επιλεγμένο κριτήριο τερματισμού (1.4), την ίδια λύση σε τέσσερα εγγυημένα δεκαδικά ψηφία. Ωστόσο, είναι εμφανές ότι η μέθοδος Jacobi είναι η πιο «αργή» από τις παραπάνω μεθόδους (απαιτεί τον μεγαλύτερο αριθμό επαναλήψεων προς επίτευξη της απαιτούμενης ακρίβειας), ενώ η μέθοδος Gauss – Seidel βελτιώνει σημαντικά, ως προς την ταχύτητα σύγκλισης, την μέθοδο Jacobi. Ταχύτετη μέθοδος από τις υλοποιούμενες, είναι εμφανώς η μέθοδος Διαδοχικής Χαλάρωσης (SOR), η οποία απαιτεί τις λιγότερες επαναλήψεις προς επίτευξη της ζητούμενης ακρίβειας, και μάλιστα η τελική λύση διαφέρει από την (θεωρούμενη ως «απόλυτα ορθή») λύση του τελεστή $A \setminus b'$ της Matlab λιγότερο από την επιλεγμένη ακρίβεια ϵ (!). ■

Άσκηση 1.3

Υλοποιήστε στην γλώσσα προγραμματισμού Matlab την επίλυση του συστήματος της άσκησης Exercise set 10.2 : 2(b) (σελίδα 644), ήτοι το σύστημα:

$$\begin{aligned}x_1^2 + x_2 - 37 &= 0 \\x_1 - x_2^2 - 5 &= 0 \\x_1 + x_2 + x_3 - 3 &= 0\end{aligned}$$

με την μέθοδο Newton – Raphson. Μπορείτε να θεωρήσετε ως μία αρχική εκτίμηση της λύσης την $\mathbf{x}^{(0)} = \mathbf{0}$.

Λύση

Προς επίλυση του προβλήματος, μετασχηματίζουμε το δεδομένο σύστημα σε διανυσματική μορφή, όπως απαιτείται για εφαρμογή της μεθόδου Newton – Raphson. Συγκεκριμένα γράφουμε το σύστημα ως:

$$\mathbf{F}(\mathbf{x}) = \mathbf{0}, \quad \mathbf{F}: \mathbb{R}^3 \rightarrow \mathbb{R}^3, \quad \mathbf{x} \in \mathbb{R}^3 \quad (1.11)$$

όπου ορίζουμε:

$$\mathbf{F}(\mathbf{x}) := \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ f_3(\mathbf{x}) \end{bmatrix} \equiv \begin{bmatrix} x_1^2 + x_2 - 37 \\ x_1 - x_2^2 - 5 \\ x_1 + x_2 + x_3 - 3 \end{bmatrix}, \quad \mathbf{x} \equiv \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \in \mathbb{R}^3 \quad (1.12)$$

Ως συνθήκη τερματισμού, επιλέγεται για την υπό μελέτη μέθοδο η εξής:

$$\frac{\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_{\infty}}{\|\mathbf{x}^{(k+1)}\|_{\infty}} < \varepsilon = 10^{-5} \quad (1.13)$$

όπου $\|\cdot\|_{\infty}$ η νόρμα μεγίστου στοιχείου, και η παράμετρος τερματισμού (ή, ακρίβεια) επιλέγεται στο 5^ο σημαντικό ψηφίο ($\varepsilon = 10^{-5}$). Η ακρίβεια επιλέγεται τέτοια, ώστε στην προεπιλεγμένη επιλογή (format) της Matlab, η οποία εμφανίζει τα αποτελέσματα με μόνο 4 δεκαδικά ψηφία, να εμφανίζονται μόνον εγγυημένα ψηφία. Με την αντίστοιχη εντολή `format long` στο τερματικό της Matlab θα εμφανιστούν και τα μη εγγυημένα ψηφία των αποτελεσμάτων που προκύπτουν από εφαρμογή της μεθόδου προς επίλυση του δεδομένου συστήματος.

(i) Μέθοδος Newton – Raphson με αριθμητικό υπολογισμό του Ιακωβιανού πίνακα της συνάρτησης του συστήματος, μέσω πεπερασμένων διαφορών.

Υλοποιείται σε γλώσσα Matlab η μέθοδος Newton – Raphson, ακολουθώντας την αλγοριθμική διαδικασία των Σημειώσεων του μαθήματος. Ο πηγαίος κώδικας γράφεται, και αποθηκεύεται, σε ένα αρχείο Matlab συνάρτησης (m-function file) με όνομα “NRMethod.m”, και παρατίθεται παρακάτω. Η συνάρτηση δέχεται ως παραμέτρους την συνάρτηση, $\mathbf{F}$, του συστήματος (σε τύπο function handle), την αρχική εκτίμηση, `x_initial`, καθώς και την επιθυμητή ακρίβεια, `epsilon`, και επιστρέφει το διάνυσμα `x_final`, ήτοι την λύση του συστήματος υπό την επιλεγμένη συνθήκη τερματισμού, καθώς και τον αριθμό επαναλήψεων, `iter`, που απαιτήθηκαν προς επίτευξή της. Αρχικά, μετά τον ορισμό της διάστασης του συστήματος (μεταβλητή `n`),

γίνεται έλεγχος της ορθότητας μορφής των δεδομένων εισόδου. Συγκεκριμένα, ελέγχεται ότι η παράμετρος ακρίβειας είναι θετική, και το διάνυσμα $x_initial$ μετατρέπεται (αν δεν είναι ήδη) σε μορφή στήλης. Επιπλέον, ελέγχεται το κατά πόσο το εισαγόμενο function handle, F , περιέχει σωστό αριθμό συναρτήσεων εντός του, με έναν βρόγχο τύπου `while(true)`. Εντός του βρόγχου δοκιμάζεται (`try`) η εύρεση του μήκους του διανύσματος που προκύπτει από εισαγωγή στο function handle ενός ψευδοτυχαίου διανύσματος μήκους n . Η εντολή $F(rand(n,1))$ καταλήγει σε σφάλμα (exception) μόνον στην περίπτωση που δεν υπάρχει μία συνάρτηση εντός του function handle που να αντιστοιχεί σε καθεμία από τις n συνιστώσες του διανύσματος $rand(n,1)$. Σε αυτή την περίπτωση, το `try` αγνοείται και στο χωρίο του `catch` το πρόγραμμα σταματά με την εντολή `error()`. Σε περίπτωση που το $F(rand(n,1))$ δεν καταλήξει σε σφάλμα (exception), το function handle είναι καλώς ορισμένο, και συμβατό με το μήκος του $x_initial$.

Μετά τον έλεγχο ορθότητας των δεδομένων εισόδου, η συνάρτηση υλοποιεί την μέθοδο Newton – Raphson. Προς αυτό τον σκοπό, αρχικοποιούνται οι «προσωρινές» (temporary) μεταβλητές για τον επαναληπτικό βρόγχο της μεθόδου. Συγκεκριμένα, αρχικοποιείται σε μηδενική τιμή (`zeros(n,n)`) ένας «προσωρινός» πίνακας, J_temp , ο οποίος κάθε φορά θα αποτελεί τον (αριθμητικά υπολογισμένο) Ιακωβιανό πίνακα του συστήματος, και ένα «προσωρινό» διάνυσμα x_temp στην τιμή του διανύσματος $x_initial$ της αρχικής εκτίμησης. Σκοπός του διανύσματος x_temp είναι σε αυτό να αντιγράφονται κάθε φορά τα αποτελέσματα της «περασμένης» επανάληψης της μεθόδου. Αρχικοποιείται επίσης το διάνυσμα εξόδου, x_final , το οποίο, για κάθε επανάληψη, αποτελεί το «τελικό» αποτέλεσμα της μεθόδου (η αρχικοποίηση του x_final είναι αυθαίρετη, εφόσον δύναται να αλλάξει ήδη από την πρώτη επανάληψη). Επιπλέον, αρχικοποιείται ο αριθμός επαναλήψεων στην τιμή 0, καθώς και η παράμετρος τερματισμού, `terminationRatio`, η οποία αρχικά τίθεται αυθαίρετα σε μία τιμή μεγαλύτερη της εισαγόμενης ακρίβειας, `epsilon`, καθώς και η παράμετρος βήματος, h , που θα αξιοποιηθεί στον αριθμητικό υπολογισμό του Ιακωβιανού πίνακα (η τιμή της h επιλέγεται όσο το δυνατόν μικρότερη, ανάλογα με την υπολογιστική ακρίβεια `eps` της Matlab – στον κώδικα τίθεται περίπου ίση με $\sqrt{eps} \sim 10^{-8}$).

Μετά την αρχικοποίηση των μεταβλητών εξόδου και των «προσωρινών» μεταβλητών, υλοποιείται ο κύριος βρόγχος `while` της μεθόδου. Συγκεκριμένα, όσο η λογική συνθήκη του βρόγχου `while` παραμένει αληθής, το πρόγραμμα υπολογίζει, αρχικά, την τιμή της συνάρτησης F στο «τρέχον» σημείο x_temp , και φροντίζει, ώστε το αποτέλεσμα που προκύπτει είναι διάνυσμα στήλης (τα function handles μπορούν να οριστούν τόσο ως «στήλες συναρτήσεων» όσο και ως «γραμμές συναρτήσεων», και το πρόγραμμα πρέπει να ανταποκρίνεται και στις δύο περιπτώσεις). Στην συνέχεια, πραγματοποιείται αριθμητικός υπολογισμός του Ιακωβιανού πίνακα του συστήματος, ο οποίος για κάθε επανάληψη αποθηκεύεται στον πίνακα J_temp . Η μέθοδος πεπερασμένων διαφορών υλοποιείται με έναν βρόγχο `for`, ο οποίος «σαρώνει» τον Ιακωβιανό πίνακα του συστήματος κατά στήλες, βάσει της σχέσης:

$$J_F(\mathbf{x}) := \left[\frac{\partial f_i(\mathbf{x})}{\partial x_j} \right]_{n \times n} \equiv \left[\frac{\partial \mathbf{F}(\mathbf{x})}{\partial x_j} \right]_{1 \times n} = \left[\frac{\partial \mathbf{F}(\mathbf{x})}{\partial x_1} \mid \dots \mid \frac{\partial \mathbf{F}(\mathbf{x})}{\partial x_n} \right]_{1 \times n} \quad (1.14)$$

Σε κάθε επανάληψη, j , του βρόγχου, υπολογίζεται μία μεταβολή κατά h στην τρέχουσα μεταβλητή x_temp κατά την διεύθυνση της j -οστής μεταβλητής, λαμβάνοντας μία πολύ μικρή μεταβολή στις τιμές της συνάρτησης F . Η τιμή της F στο μετατοπισμένο σημείο xh_step αποθηκεύεται στο διάνυσμα στήλη Fh_step , και στην συνέχεια η παράγωγος της F κατά την j -οστή μεταβλητή προσεγγίζεται από την σχέση:

$$\frac{\partial F(x)}{\partial x_j} \approx \frac{F(x_1, \dots, x_j + h, \dots, x_n) - F(x_1, \dots, x_j, \dots, x_n)}{h} \rightarrow \frac{Fh_step - F_temp}{h}$$

η οποία ορίζει και την μέθοδο πεπερασμένων διαφορών. Πριν τον τερματισμό κάθε «εξωτερικού» βρόγχου `while`, ελέγχεται εάν ο Ιακωβιανός πίνακας είναι ιδιάζων (περίπτωση κατά την οποία η μέθοδος εξέρχεται του βρόγχου, και λαμβάνεται αντίστοιχο μήνυμα), ενώ ο αριθμός επαναλήψεων αυξάνεται κατά 1, η παράμετρος τερματισμού, `terminationRatio`, τίθεται στην νέα τιμή (από την σχέση (1.13) παραπάνω), και η «τρέχουσα» λύση, `x_temp`, ανατίθεται στην λύση της «προηγούμενης» επανάληψης, `x_final`. Ο πλήρης πηγαίος κώδικας που υλοποιεί τα παραπάνω παρουσιάζεται παρακάτω.

MATLAB FUNCTION M-FILE (NRMethod.m):

```
function [x_final, iter] = NRMethod(F, x_initial, epsilon)
% Function NRMethod.m
%-----
% Newton-Raphson's method for non-linear system F(x) = 0.
% (--- The Jacobian matrix is calculated numerically---)
%
% Input : - function handle F for the system of equations.
%         - initial guess x_initial          (n x 1).
%         - termination parameter epsilon    (1 x 1).
%         (use of "weighted" infinity norm)
% Output: - vector x_final = sol. of (F(x)=0) (n x 1).
%         - integer iter = no. of iterations (1 x 1).
%-----
% --- System dimension definition ---
n = length(x_initial);

% --- Input validity ---
% Ensure epsilon is positive:
if(epsilon <= 0)
    error('Termination parameter epsilon must be positive.');
```

```
end
% Ensure x_initial is in column form :
x_initial = x_initial(:);
% Ensure F is properly sized :
while(true)
    try
        F(rand(n,1));
        break;
    catch
        error('Function handle F and x_initial are incompatible.');
```

```
end
end
% --- Newton-Raphson method implementation ---
```

```
% -----
% --- Temporary and Input variables initialization ---
% Initializations:
J_temp = zeros(n,n);
x_temp = x_initial;
x_final = x_temp;
iter = 0;
% Termination Ratio ("Weighted" infinity norm):
terminationRatio = 1 + epsilon;
% Perturbation step size for finite differences:
h = 1e-8;
% -----
% --- Main Iterative Loop ---
while(terminationRatio >= epsilon)
    % --- 1. Calculation of F(x(k)) ---
    F_temp = F(x_temp);
    % Ensure column form (F is a function handle):
    F_temp = F_temp(:);
    % --- 2. Calculation of J(x(k)) ---
    for j = 1 : n
        xh_step = x_temp;
        xh_step(j) = x_temp(j) + h;    % Perturbation of j-th variable.
        Fh_step = F(xh_step);
        Fh_step = Fh_step(:);          % Ensure column form.
        % Finite difference formula [ dFi/dxj ~ (Fi(xj+h)-Fi(xj))/h ]:
        J_temp(:,j) = (Fh_step - F_temp)/h;
    end
    % --- 3. Update solution and temp. parameters ---
    % Check if J_temp is singular:
    if(det(J_temp) == 0)
        disp('Method terminated because J became singular. ');
        break;
    end
    % F_temp is ensured to be in column form, so:
    x_final = x_temp - J_temp \ F_temp;
    terminationRatio = norm(x_final-x_temp, "inf") / norm(x_final, "inf");
    iter = iter + 1;
    x_temp = x_final;
end
% -----
end
```

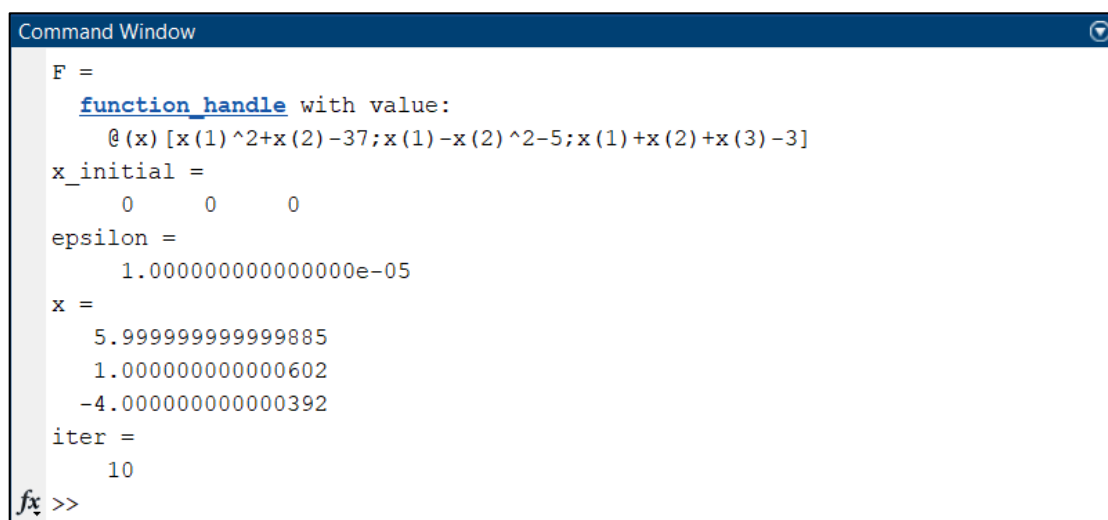
Βάσει της παραπάνω συνάρτησης, η επίλυση του συστήματος γίνεται τετριμμένη, και παρόλο που μπορεί να γίνει απευθείας στο Command Window, παρουσιάζεται για εποπτικούς λόγους στο παρακάτω αρχείο (m-script file).

MATLAB SCRIPT M-FILE (NRMethodEx1_3_Script.m):

```
clear
clc

% --- CFD Exercise 1.3 ---
% Solution script (using NRMethod.m)
%
% --- Definition of function handle ---
F = @(x) [x(1)^2+x(2)-37;
          x(1)-x(2)^2-5;
```

```
x(1)+x(2)+x(3)-3]
%
% --- Definition of x_initial ---
x_initial = [0 0 0]
%
% --- Definition of accuracy epsilon ---
epsilon = 1e-5
% --- Apply Newton-Raphson method and get solution ---
[x,iter] = NRMethod(F, x_initial, epsilon)
% End of script.
```



```
Command Window

F =
    function_handle with value:
    @(x) [x(1)^2+x(2)-37;x(1)-x(2)^2-5;x(1)+x(2)+x(3)-3]
x_initial =
     0     0     0
epsilon =
    1.000000000000000e-05
x =
    5.999999999999885
    1.000000000000602
   -4.000000000000392
iter =
     10
fx >>
```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του δεδομένου συστήματος με την μέθοδο Newton – Raphson, όπως φαίνεται παραπάνω (σε μορφή εκτύπωσης `format long`; `format compact`). Λαμβάνουμε επίσης τον αριθμό επαναλήψεων που απαιτήθηκαν από την εν λόγω μέθοδο για επίτευξη της απαιτούμενης ακρίβειας της λύσης του δεδομένου συστήματος.

Βάσει των παραπάνω, συμπεραίνουμε ότι η λύση του δεδομένου συστήματος, όπως προέκυψε με την επίλυσή του κάνοντας χρήση της μεθόδου Newton – Raphson είναι η εξής:

$$\mathbf{x}_{\text{NR}} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6.0000 \\ 1.0000 \\ -4.0000 \end{bmatrix} \equiv \begin{bmatrix} 6 \\ 1 \\ -4 \end{bmatrix}, \quad N_{\text{iterations}}^{\text{NR}} = 10, \quad \varepsilon = 10^{-5} \quad (1.15)$$

όπου τα αποτελέσματα αναγράφονται με τέσσερα εγγυημένα δεκαδικά ψηφία (όπως λαμβάνονται υπό την εντολή `format default` στο Command Window του Matlab), όπως επιβάλλει η επιλεγόμενη ακρίβεια. Στην τελευταία «ισότητα» εμφανίζεται η λύση του συστήματος, όπως προκύπτει «αναλυτικά» μέσω επίλυσής του «δια χειρός». Είναι φανερό ότι η συνάρτηση που υλοποιήθηκε παραπάνω λειτουργεί σωστά, και αποδίδει ορθά αποτελέσματα. Επιπλέον, σημειώνεται στο παραπάνω αποτέλεσμα και ο αριθμός των επαναλήψεων που απαιτήθηκαν ώστε να ληφθεί η συγκεκριμένη ακρίβεια στην λύση του δεδομένου συστήματος. Σημειώνεται, ακόμα, ότι η μέθοδος N-R καταφέρνει να προσεγγίσει την λύση σε αρκετά περισσότερα δεκαδικά ψηφία από τον αριθμό αυτών που επιβάλλεται από την επιλεγόμενη ακρίβεια `epsilon`.

(ii) Μέθοδος Newton – Raphson με αναλυτικό υπολογισμό του Ιακωβιανού πίνακα της συνάρτησης του συστήματος, μέσω δια χειρός παραγωγίσεων.

Υλοποιείται σε γλώσσα Matlab η μέθοδος Newton – Raphson, ακολουθώντας την αλγοριθμική διαδικασία που ακολουθείται και στο εδάφιο (i) της παρούσας άσκησης. Ο κώδικας που υλοποιεί την μέθοδο Newton – Raphson είναι εντελώς όμοιος με αυτόν της προηγούμενης περίπτωσης, με την διαφορά ότι πλέον ο Ιακωβιανός πίνακας της συνάρτησης του συστήματος εισάγεται, υπό μορφή function handle, ως όρισμα της συνάρτησης. Επομένως, ο χρήστης, εκτός του function handle της συνάρτησης του συστήματος, πρέπει να έχει προσδιορίσει εκπεφρασμένα και τον Ιακωβιανό πίνακα αυτής, ως function handle. Πριν την απευθείας παρουσίαση του αντίστοιχου κώδικα, αξιωματικώς σημειωθεί ότι στην συνάρτηση του δεδομένου συστήματος, σχέση (1.12), αντιστοιχεί ο Ιακωβιανός πίνακας:

$$\mathbf{J}_F(\mathbf{x}) \equiv \left[\frac{\partial f_i(\mathbf{x})}{\partial x_j} \right]_{3 \times 3} = \begin{bmatrix} 2x_1 & 1 & 0 \\ 1 & -2x_2 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad \forall \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \in \mathbb{R}^3 \quad (1.16)$$

βάσει της οποίας κατασκευάζεται το αντίστοιχο function handle που εισάγεται ως όρισμα στην συνάρτηση m-function που υλοποιεί την Μέθοδο Newton – Raphson. Ο πλήρης πηγαίος κώδικας που υλοποιεί τα παραπάνω παρουσιάζεται παρακάτω.

MATLAB FUNCTION M-FILE (NRMethod_Jin.m):

```
function [x_final, iter] = NRMethod_Jin(F, J, x_initial, epsilon)
% Function NRMethod_Jin.m
%-----
% Newton-Raphson's method for non-linear system F(x) = 0.
% (--- The Jacobian matrix is given analytically ---)
%
% Input : - function handle F for the system of equations.
%         - function handle J for the Jacobian matrix.
% (Warning: J must be given in the right 3x3 form by the user!)
%         - initial guess x_initial          (n x 1).
%         - termination parameter epsilon    (1 x 1).
%         (use of "weighted" infinity norm)
% Output: - vector x_final = sol. of (F(x)=0) (n x 1).
%         - integer iter = no. of iterations (1 x 1).
%-----

% --- System dimension definition ---
n = length(x_initial);

% --- Input validity ---
% Ensure epsilon is positive:
if(epsilon <= 0)
    error('Termination parameter epsilon must be positive.');
```

```
        catch
            error('Function handle F and x_initial are incompatible.');
```

```
        end
    end
while(true)
    try
        J(rand(n,n));
        break;
    catch
        error('Function handle J and x_initial are incompatible.');
```

```
    end
end

% --- Newton-Raphson method implementation ---
% -----
% --- Temporary and Input variables initialization ---
% Initializations:
x_temp = x_initial;
x_final = x_temp;
iter = 0;
% Termination Ratio ("Weighted" infinity norm):
terminationRatio = 1 + epsilon;
% -----
% --- Main Iterative Loop ---
while(terminationRatio >= epsilon)
    % --- 1. Calculation of F(x(k)) ---
    F_temp = F(x_temp);
    % Ensure column form (F is a function handle):
    F_temp = F_temp(:);
    % --- 2. Calculation of J(x(k)) ---
    J_temp = J(x_temp);
    % --- 3. Update solution and temp. parameters ---
    % Check if J_temp is singular:
    if(det(J_temp) == 0)
        disp('Method terminated because J became singular.');
```

```
        break;
    end
    % F_temp is ensured to be in column form, so:
    x_final = x_temp - J_temp\F_temp;
    terminationRatio = norm(x_final-x_temp, "inf")/norm(x_final,"inf");
    iter = iter + 1;
    x_temp = x_final;
end
% -----
end
```

Βάσει της παραπάνω συνάρτησης, η επίλυση του συστήματος γίνεται τετριμμένη, και παρόλο που μπορεί να γίνει απευθείας στο Command Window, παρουσιάζεται για εποπτικούς λόγους στο παρακάτω αρχείο (m-script file).

Αξίζει να σημειωθεί ότι με την παραπάνω υλοποίηση της μεθόδου, επαφίεται πλήρως στον χρήστη της συνάρτησης η ορθή εισαγωγή του Ιακωβιανού πίνακα του συστήματος (χωρίς να μπορεί να ελεγχθεί με κάποιο τρόπο, λόγω του τρόπου λειτουργίας των function handles στην Matlab), επομένως, μπορούν να συμβούν αρκετές λάθος χρήσεις της συνάρτησης, χωρίς εποπτεία των προβληματικών σημείων των δεδομένων εισόδου (πιθανώς σχετικά δύσκολο debugging). Επιπλέον, σε πολλές

περιπτώσεις η αναλυτική εύρεση του Ιακωβιανού πίνακα της συνάρτησης του συστήματος είναι δυσχερής, ή και αδύνατη, επομένως η εύρεση του Ιακωβιανού πίνακα είναι προτιμότερο, και πιο γενικό, να πραγματοποιηθεί αριθμητικά, όπως στο παραπάνω εδάφιο (i) της παρούσας άσκησης. Στο παρακάτω αρχείο m-script, γίνεται επίλυση του συστήματος και με την συνάρτηση που υλοποιήθηκε στο εδάφιο (i) παραπάνω, προς επαλήθευση της λύσης, σχολιασμό και σύγκριση.

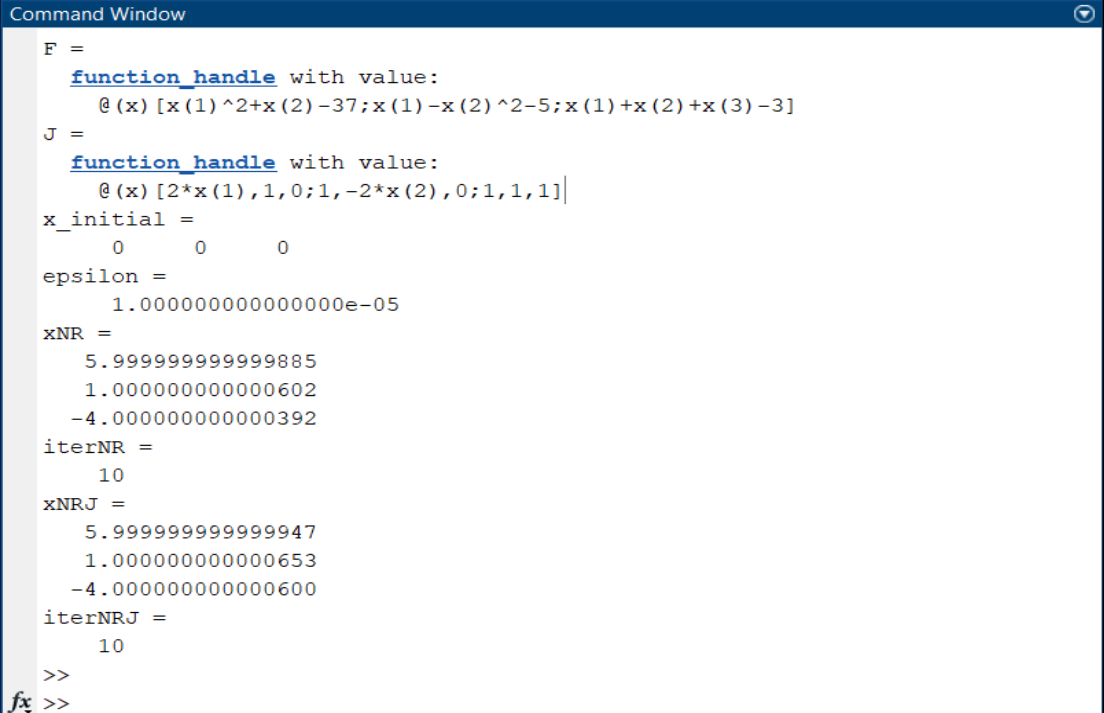
MATLAB SCRIPT M-FILE (NRMethodEx1_3_Script.m):

```
clear
clc

% --- CFD Exercise 1.3 ---
% Solution script (using NRMethod.m , NRMethod_Jin.m)
%
% --- Definition of function handle F ---
F = @(x) [x(1)^2+x(2)-37;
          x(1)-x(2)^2-5;
          x(1)+x(2)+x(3)-3]

%
% --- Definition of function handle J ---
J = @(x) [2*x(1),1,0; 1,-2*x(2),0; 1,1,1]

% --- Definition of x_initial ---
x_initial = [0 0 0]
%
% --- Definition of accuracy epsilon ---
epsilon = 1e-5
% --- Apply Newton-Raphson method and get solution ---
[xNR,iterNR] = NRMethod(F, x_initial, epsilon)
% --- Apply Newton-Raphson (with J) method and get solution ---
[xNRJ,iterNRJ] = NRMethod_Jin(F, J, x_initial, epsilon)
% End of script.
```



```
Command Window

F =
    function_handle with value:
        @(x) [x(1)^2+x(2)-37;x(1)-x(2)^2-5;x(1)+x(2)+x(3)-3]

J =
    function_handle with value:
        @(x) [2*x(1),1,0;1,-2*x(2),0;1,1,1]

x_initial =
     0     0     0

epsilon =
    1.0000000000000000e-05

xNR =
     5.999999999999885
     1.0000000000000602
    -4.0000000000000392

iterNR =
     10

xNRJ =
     5.999999999999947
     1.0000000000000653
    -4.0000000000000600

iterNRJ =
     10

>>
fx >>
```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του δεδομένου συστήματος με την μέθοδο Newton – Raphson (και με τις δύο διαφορετικές υλοποιήσεις της), όπως φαίνεται παραπάνω (σε μορφή εκτύπωσης `format long`; `format compact`). Λαμβάνουμε επίσης τον αριθμό επαναλήψεων που απαιτήθηκαν από την εν λόγω μέθοδο για επίτευξη της απαιτούμενης ακρίβειας της λύσης του δεδομένου συστήματος.

Βάσει των παραπάνω, συμπεραίνουμε ότι η λύση του δεδομένου συστήματος, όπως προέκυψε με την επίλυσή του κάνοντας χρήση της μεθόδου Newton – Raphson, μέσω αναλυτικού υπολογισμού του Ιακωβιανού πίνακα του συστήματος, είναι η εξής:

$$\mathbf{x}_{\text{NRJ}} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6.0000 \\ 1.0000 \\ -4.0000 \end{bmatrix} \equiv \begin{bmatrix} 6 \\ 1 \\ -4 \end{bmatrix}, \quad N_{\text{iterations}}^{\text{NRJ}} = 10, \quad \varepsilon = 10^{-5} \quad (1.15)$$

όπου τα αποτελέσματα αναγράφονται με τέσσερα *εγγυημένα* δεκαδικά ψηφία (όπως λαμβάνονται υπό την εντολή `format default` στο Command Window του Matlab), όπως επιβάλλει η επιλεγόμενη ακρίβεια. Στην τελευταία «ισότητα» εμφανίζεται η λύση του συστήματος, όπως προκύπτει «αναλυτικά» μέσω επίλυσής του «δια χειρός». Είναι φανερό ότι η συνάρτηση που υλοποιήθηκε παραπάνω λειτουργεί σωστά, και αποδίδει ορθά αποτελέσματα. Επιπλέον, σημειώνεται στο παραπάνω αποτέλεσμα και ο αριθμός των επαναλήψεων που απαιτήθηκαν ώστε να ληφθεί η συγκεκριμένη ακρίβεια στην λύση του δεδομένου συστήματος. Σημειώνεται, ακόμα, ότι η μέθοδος Newton – Raphson καταφέρνει να προσεγγίσει την λύση σε αρκετά περισσότερα δεκαδικά ψηφία από τον αριθμό αυτών που επιβάλλεται από την επιλεγόμενη ακρίβεια `epsilon`. ■